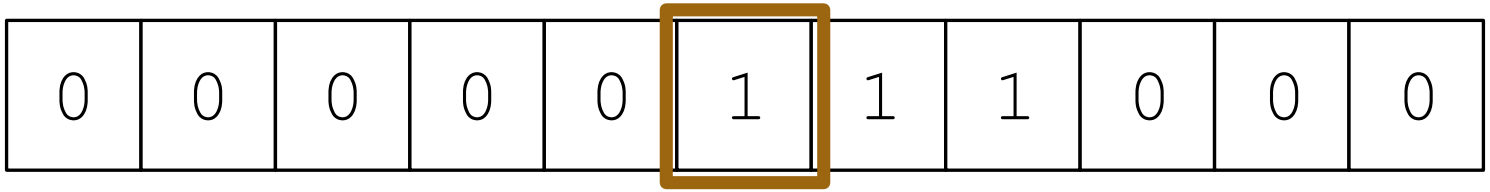
TURING MACHINES



“COMPUTER” USED TO BE A JOB

- Before electronic computers, a computer was a person.
- Computers did long calculations by hand, following written instructions step by step.
- They didn’t need to understand why the steps worked. They just needed to follow them carefully.

Instructions

written steps to follow

Paper and pencil

to write things down

Their memory

of which step they’re on

What a human computer worked with

NASA computers, mid-1950s



Human computers at work at JPL, mid-1950s
(photo credit: NASA/JPL-Caltech)

Instructions

written steps to follow

Paper and pencil

to write things down

Their memory

of which step they're on

What a human computer worked with

A QUESTION FROM 1928

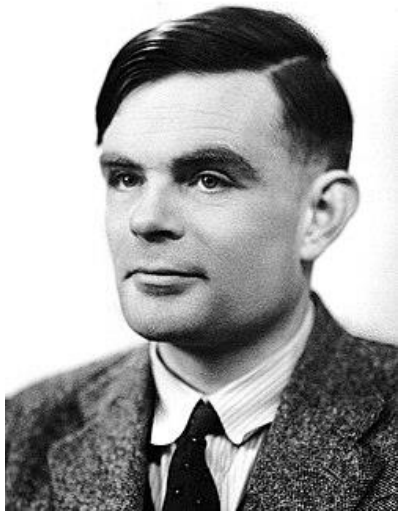
- In 1928 the mathematician David Hilbert asked a question about math itself.
- To answer “no,” someone first had to say exactly what “following steps mechanically” means.

Is there a step-by-step procedure that can take any math statement and decide whether it can be proved?

Hilbert’s question, in plain words. Its German name is the Entscheidungsproblem (“decision problem”).

ALAN TURING

- A British mathematician. In 1936 he answered Hilbert's question.
- His idea: picture a human computer, then strip the job down to the simplest possible steps.
- The result is a precise description of "following steps." Today we call it a Turing machine.



Alan Turing in 1951
(photo: Elliott & Fry. © National Portrait Gallery, London)



Computer science's biggest prize, the Turing Award, is named after him.

WHAT A HUMAN COMPUTER DOES

Turing noticed that every step of hand calculation comes down to a few things:

Looks at one symbol

You only ever look at a small part of the paper at a time.

Maybe changes it

Write a symbol, or erase one and write another.

Moves a little

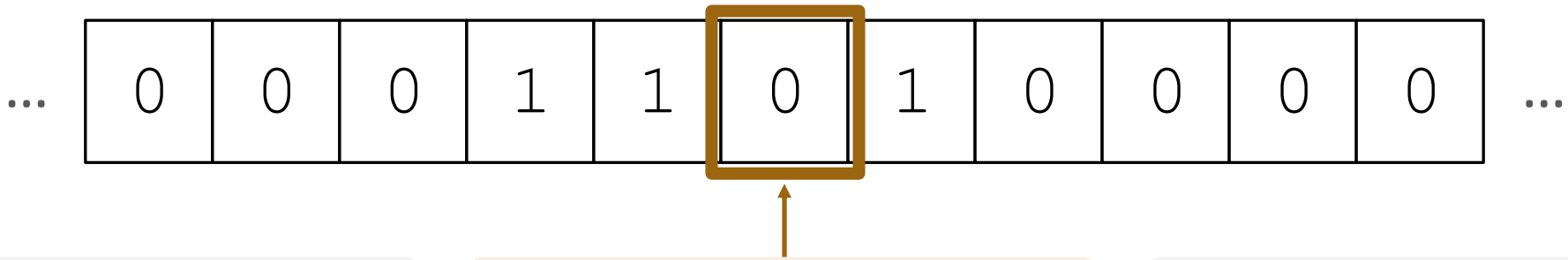
Look at the next symbol over.

Remembers where they are

“I’m on step 4 of the instructions.” Turing called this the state.

Anything more complicated can be built out of these.

THE PARTS OF A TURING MACHINE



Tape

A row of squares. Each square holds 0 or 1. The tape goes on as far as we need, in both directions.

Head

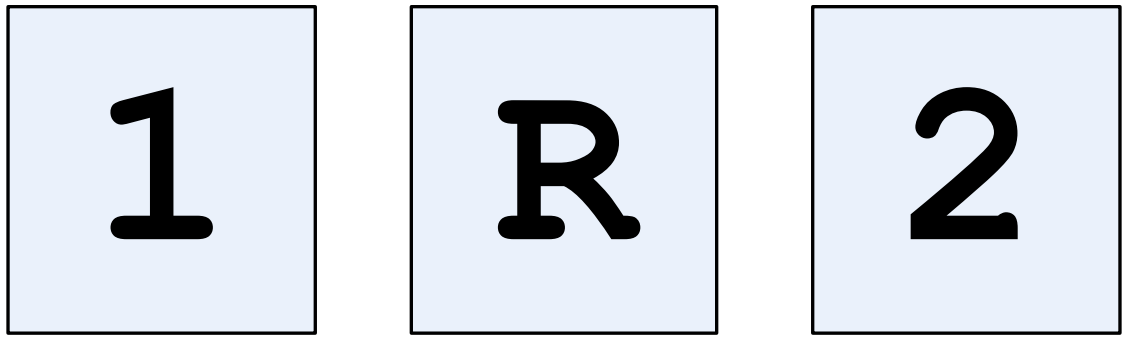
It reads the one square it's on, can write 0 or 1 there, then moves one square left or right.

State and rules

The state is a number: which step the machine is on. A table of rules says what to do in each state.

That's the whole machine.

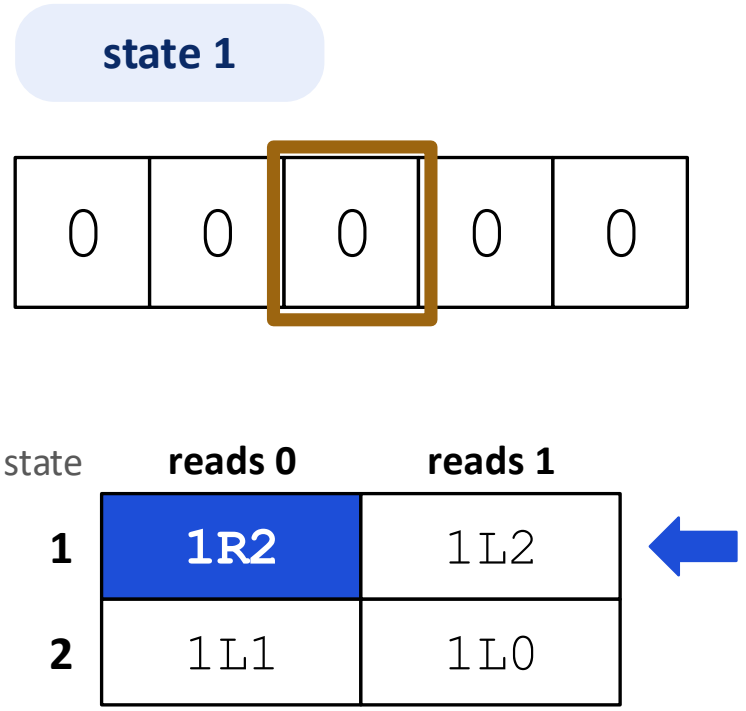
READING ONE RULE



- Every rule says three things, always in this order: what to write, which way to move, and which state to go to next.
- **`L`** means move left. **`R`** means move right.
- Going to state **0** means stop. The machine has halted.

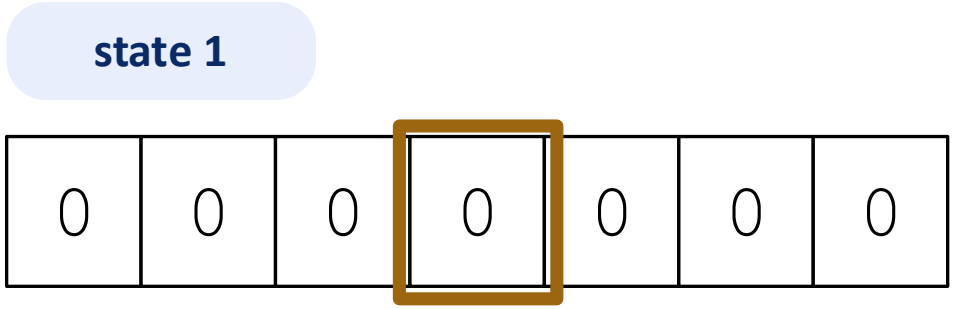
THE RULE TABLE

- One row for each state. One column for each symbol the head might read.
- To take a step, the machine finds the row for its state and the column for the symbol under the head. That cell is the rule to use.
- Here the machine is in state 1 and the head reads 0, so it uses 1R2.



THE FIRST STEP

Start: a blank tape (all 0s), the machine in state 1.

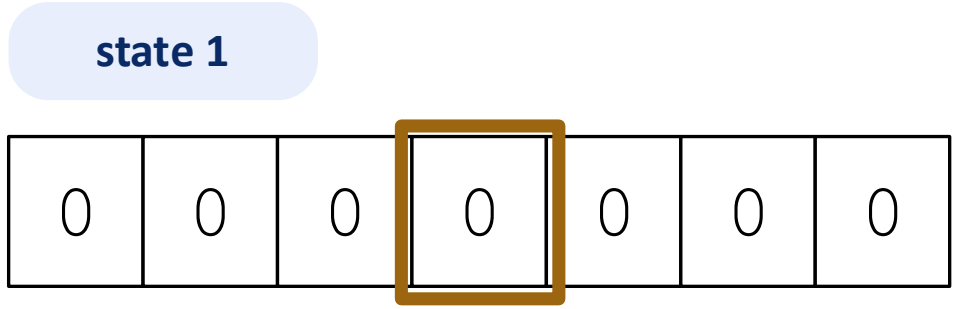


state	reads 0	reads 1
1	1R2	1L2
2	1L1	1L0

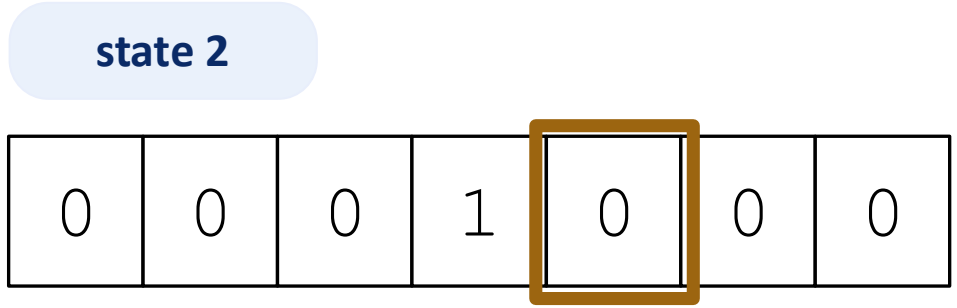
Predict: which rule does the machine use? What do the tape and state look like after one step?

THE FIRST STEP: ANSWER

Start: a blank tape (all 0s), the machine in state 1.



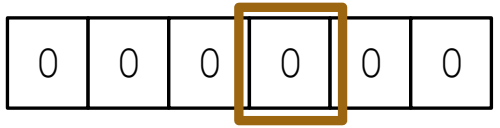
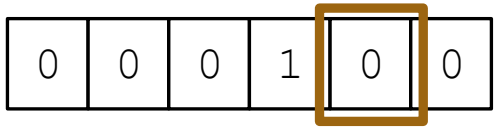
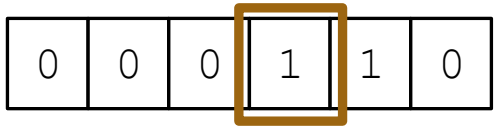
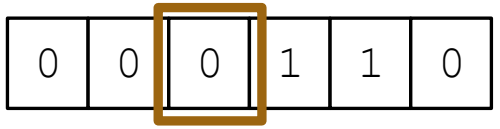
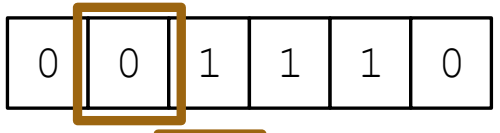
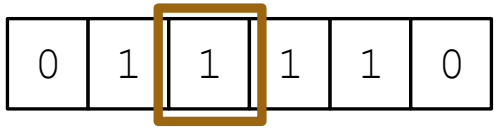
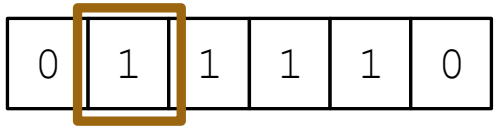
After one step:



state	reads 0	reads 1
1	**1R2**	1L2
2	1L1	1L0

- State 1, reading 0: the rule is **1R2**.
- It writes 1, moves one square right, and switches to state 2.

THE WHOLE RUN

step	state	reads	rule	tape after the step
start	1			
1	1	0	**1R2**	
2	2	0	**1L1**	
3	1	1	**1L2**	
4	2	0	**1L1**	
5	1	0	**1R2**	
6	2	1	**1L0**	

"state" is the state the machine is in when it takes that step.

Halted

Step 6 goes to state 0, so the machine stops. It left four 1s, with the head on the first one.

THE MACHINE, MORE PRECISELY

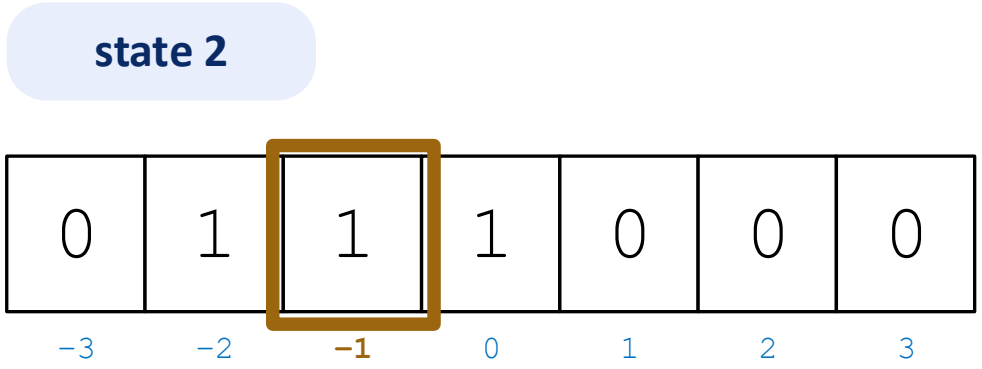
- **States:** 1, 2, …, n. The machine starts in state 1. State 0 means halt.
- **Symbols:** 0 and 1. Every square holds 0 unless the input put a 1 there.
- **Rules:** for each state and each symbol, at most one rule. A rule is three things: a symbol to write, a direction, and a next state.
- Mathematicians call the rule table the **transition function** and often write it as δ (delta).

```
δ(1, 0) = (1, R, 2)
```

In state 1, reading 0: write 1, move right, go to state 2. In our table, that's the cell 1R2.

A SNAPSHOT OF THE MACHINE

- At any moment, three things describe the whole machine: what's on the tape, which square the head is on, and the current state.
- The rules never change while the machine runs.
- Square numbers can be negative. The head can walk left of where it started.
- A snapshot like this is called a **configuration**. Each row of the whole run was one.



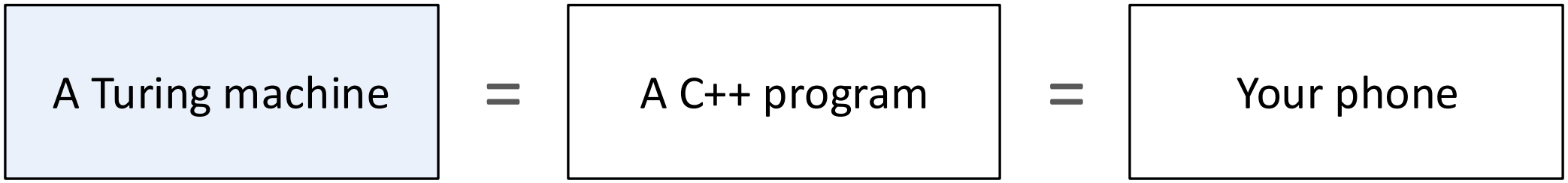
tape: 1s on squares −2 to 0. Head: square −1. State: 2.

ONE STEP, PRECISELY

1. If the state is 0, the machine has halted. Stop.
2. Read the symbol under the head.
3. Find the rule for this state and this symbol. If that cell is blank, stop with an error.
4. Write the rule's symbol on the square under the head.
5. Move the head one square left or right.
6. Switch to the rule's next state. Go back to 1.

The order matters. **`1L0`** writes 1, moves left, and then halts, so the head finishes one square to the left of where it wrote.

WHY SUCH A SIMPLE MACHINE MATTERS

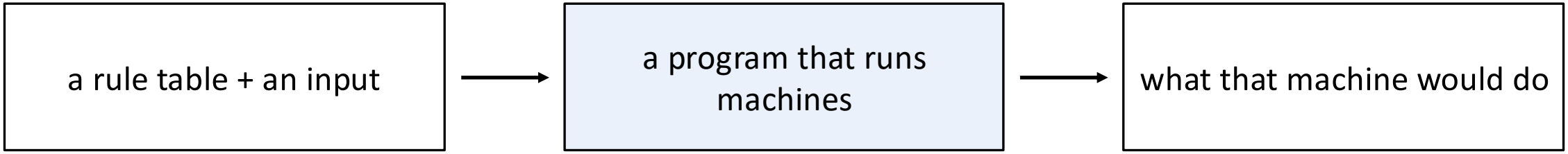


can compute the same things, given enough time and tape

- As far as anyone knows, anything any computer can calculate, a Turing machine can calculate too.
- A real computer is much faster and much easier to program. But it can’t compute anything a Turing machine can’t.
- This idea is called the Church–Turing thesis. Alonzo Church reached the same conclusion in 1936 in a different way.

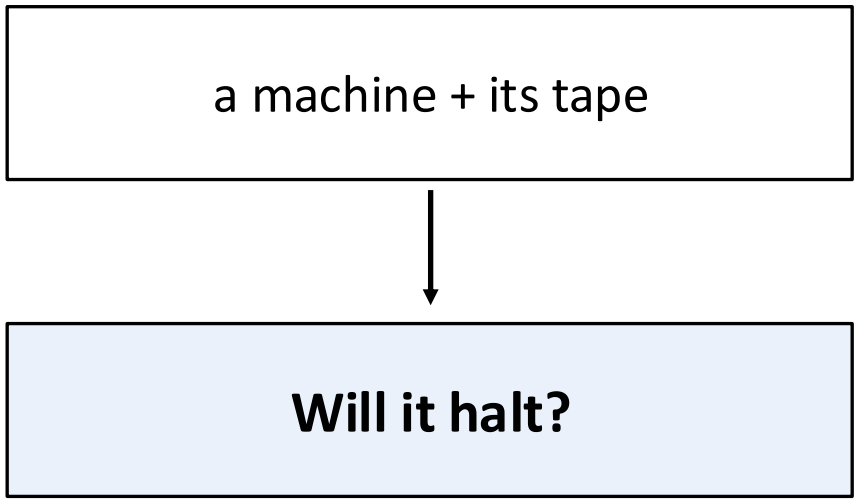
A MACHINE THAT RUNS MACHINES

- Turing's paper also described a universal machine. Give it another machine's rule table and that machine's input, and it does whatever that machine would do.
- Every computer you've used works this way: a program is data that another program reads and follows.
- Our simulator is a program that runs Turing machines. It's written in JavaScript. Later in this unit, you'll write one in C++.



SOMETHING NO MACHINE CAN DO

- Some machines halt. Others run forever.
- Turing proved that no machine can take any machine plus its starting tape and always say correctly whether it will halt.
- Today this is called the halting problem. It's how Turing answered Hilbert: no, there is no such general procedure.



No machine can always answer this correctly.

Next: Turing's argument, written as C++.

SUPPOSE WE COULD CHECK FOR HALTING

```
// Returns true if program, run on input,
// eventually stops. Always returns an answer.
bool halts(string program, string input) {
    // ... suppose someone figured this out ...
}
```

- Suppose someone wrote this function, and it always gives the right answer.
- A program is just text, so it can be passed in as a string.
- We'll show this leads to a contradiction, so the function can't exist.

A PROGRAM THAT DOES THE OPPOSITE

```
void contrary(string program) {
    if (halts(program, program)) {
        while (true) {
        }
    }
}
```

- **`contrary`** asks: would this program halt if its own text were its input?
- If yes, it loops forever.
- If no, it returns right away.

FEEDING `contrary` ITSELF

```
// me holds the source code of contrary
contrary(me);
```

Predict: does this call halt or run forever? Try both answers.

FEEDING `contrary` ITSELF: ANSWER

```
// me holds the source code of contrary
contrary(me);
```

If `halts(me, me)` says it halts…
…then `contrary` loops forever. So `halts` was wrong.

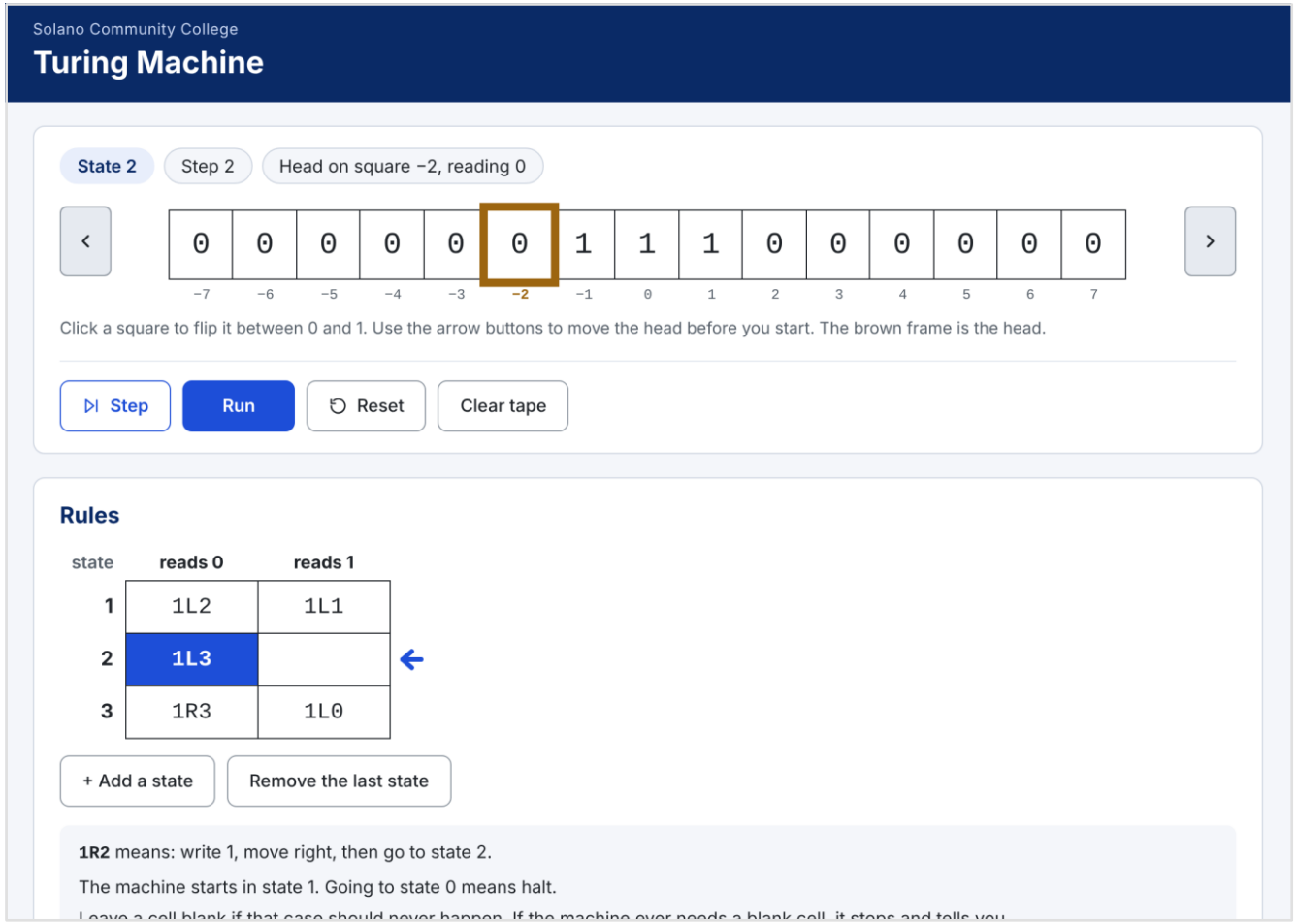
If `halts(me, me)` says it runs forever…
…then `contrary` returns right away. So `halts` was wrong.

Either way, `halts` gives a wrong answer. So a `halts` that's always right can't exist. Turing's 1936 proof uses the same trick, with machines in place of functions.

OUR TURING MACHINE

ben-allen.github.io/CIS-022/tools/turing-machine.html

THE SCREEN



1 Status

the state, the step count, and what the head is reading

2 Tape

the brown frame is the head; square numbers are underneath

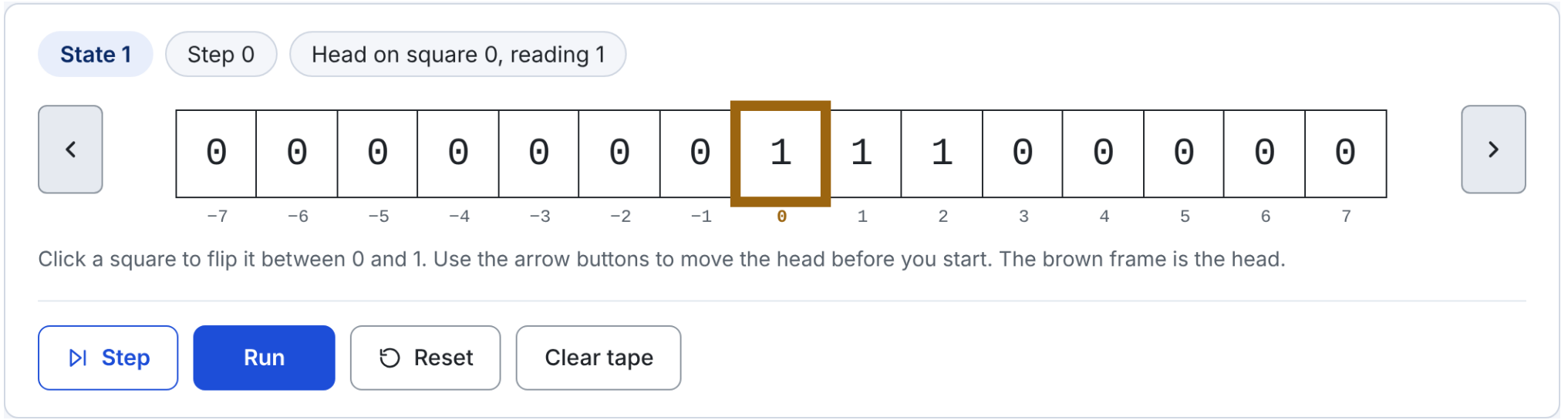
3 Buttons

Step, Run, Reset, Clear tape

4 Rules

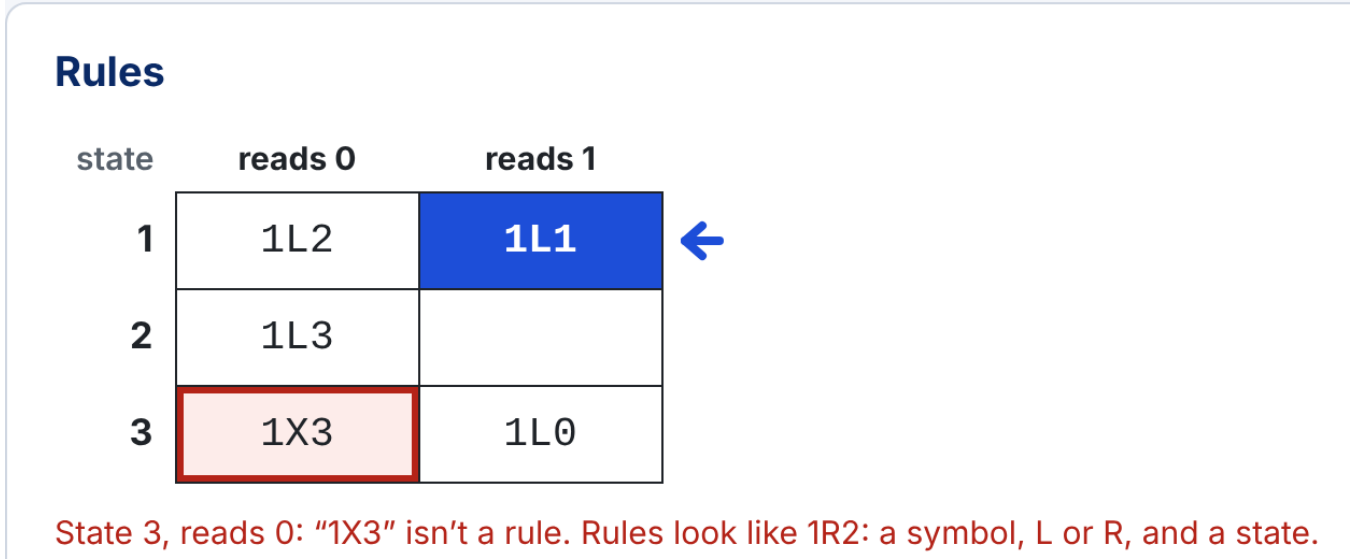
the table you type your machine into

SETTING UP THE TAPE



- Click a square to flip it between 0 and 1.
- The arrow buttons at each end move the head one square.
- **Clear tape** sets every square back to 0.
- Above: the number 3, ready to go. Three 1s in a row, head on the leftmost 1.

TYPING IN RULES



Here someone typed X where L or R should be.

- Type a rule like **`1R2`** into a cell. Lowercase is fine.
- Leave a cell blank if that case should never happen.
- **+ Add a state** adds a row.
- A red outline means the simulator can’t read that rule. The message under the table says why.

WRITING A MACHINE DOWN

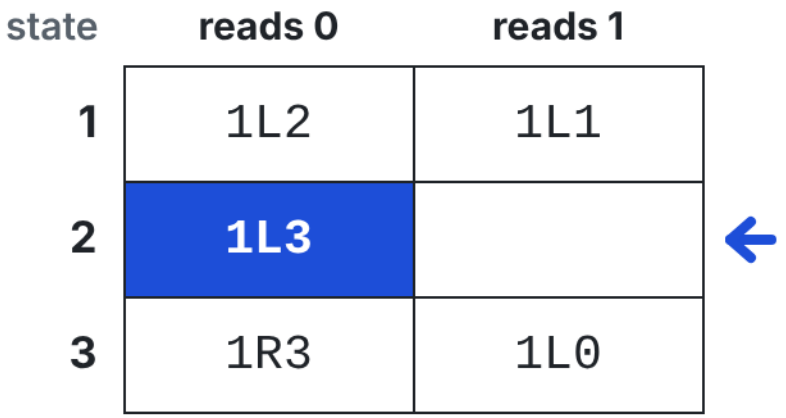
```
# today's first machine
1: 1R2 1L2
2: 1L1 1L0
```

- One line per state: the state number, a colon, the rule for reading 0, then the rule for reading 1.
- A dash (−) means a blank cell.
- Lines that start with # are comments.
- You'll turn in your machines this way, and it's the format your own C++ Turing machine will read.

RUNNING YOUR MACHINE



- **Step** uses one rule, then waits.
- **Run** keeps stepping until the machine stops. While it runs, the button says **Pause**.
- **Reset** puts back the tape you started with, in state 1.



state	reads 0	reads 1
1	1L2	1L1
2	**1L3**	
3	1R3	1L0

The blue cell is the rule the machine will use next.
The arrow marks the current state.

Stepping is the best way to find out what your machine is really doing.

WHEN IT STOPS

Halted after 5 steps. The tape holds 6, a well-formed number.

The machine reached state 0. The tape holds a number, written the way we write numbers here (next section).

Halted after 4 steps. The tape is blank.

It halted with nothing on the tape.

Stopped after 4 steps. There is no rule for state 3 reading 1. Fix the rule, then press Step or Run to carry on.

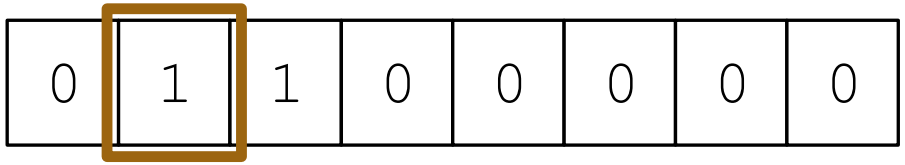
It needed a rule from a blank cell. That cell gets a red outline. Fill it in, then press Step or Run to carry on.

If Run keeps going and going, your machine may never halt. Press Pause, then Step to see what it repeats.

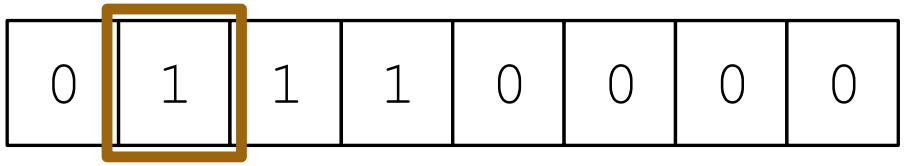
WRITING NUMBERS WITH 1s

- We write a number as that many 1s in a row. 3 is 1 1 1.
- 0s on both sides show where the number starts and stops. A blank tape is 0.
- Machines start and finish with the head on the leftmost 1. We call that well-formed.

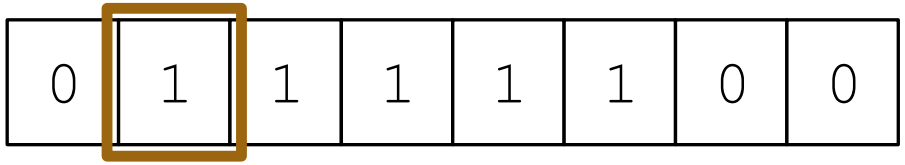
2



3



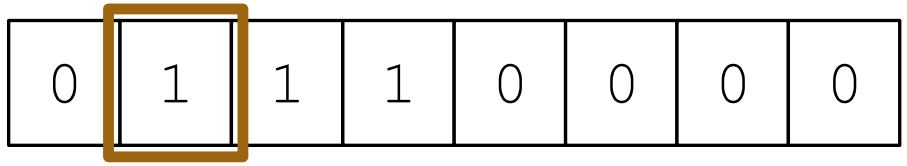
5



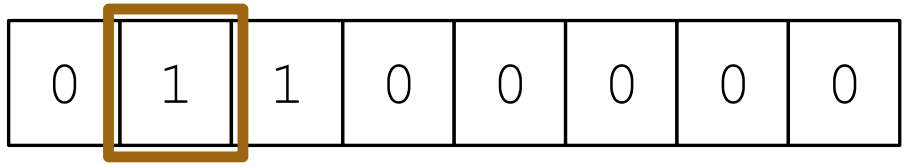
Why not binary? Machines that count in 1s are much shorter to write. If we represented numbers in binary, how would we determine where numbers start and end?

PUZZLE 1: SUBTRACT 1

Start



Finish



- Write a machine that takes away one 1.
- It should work for any number that's 1 or more.

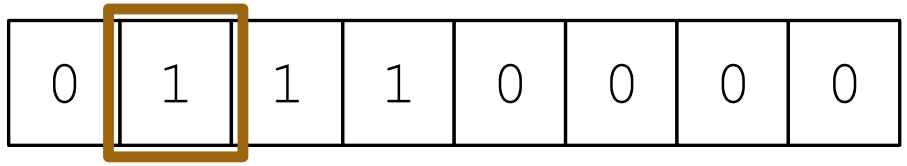
Hint: you need only one rule.

Check: put 3 on the tape and press Run. The message should end with:

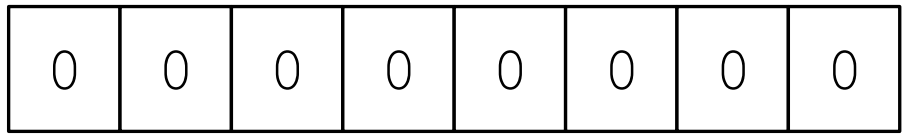
> The tape holds 2, a well-formed number.

PUZZLE 2: ERASE A NUMBER

Start



Finish



(blank tape: the head can be anywhere)

- Write a machine that erases the whole number, leaving a blank tape.
- It should work for any number, including 0.

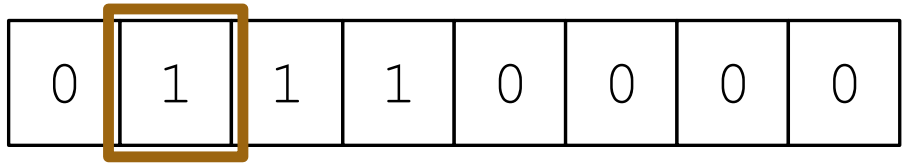
Hint: a rule can send the machine back to the state it's already in. That's how a machine repeats.

Check: put 3 on the tape and press Run. The message should end with:

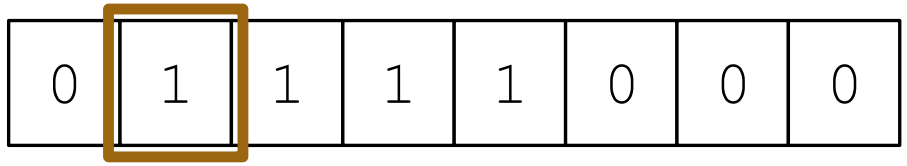
> The tape is blank.

PUZZLE 3: ADD 1

Start



Finish



- Write a machine that adds one more 1.
- Finish with the head on the leftmost 1.

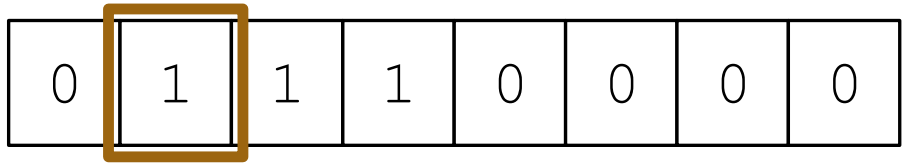
Hint: every rule moves the head. Think about where the head is after the last rule.

Check: put 3 on the tape and press Run. The message should end with:

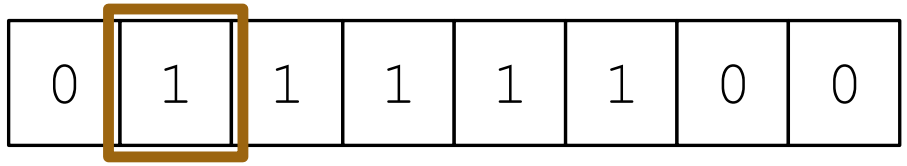
> The tape holds 4, a well-formed number.

PUZZLE 4: ADD 2

Start



Finish



- Write a machine that adds two 1s.
- Try changing your answer to puzzle 3.

Hint: test it with 0 too. Does it still finish on the leftmost 1?

Check: put 3 on the tape and press Run. The message should end with:

> The tape holds 5, a well-formed number.

WHAT DOES THIS MACHINE DO?

state	reads 0	reads 1
1	1L2	1L1
2	1L3	
3	1R3	1L0

Predict: type this machine in. Put 2 on the tape and run it. Then try 4. What does it do?

Before you run it: which state does the machine stay in while it reads 1s?

WHAT DOES THIS MACHINE DO? ANSWER

state	reads 0	reads 1
1	`1L2`	`1L1`
2	`1L3`	
3	`1R3`	`1L0`

- **It adds 3.** 2 becomes 5, and 4 becomes 7.
- State 1 moves left past the 1s (**`1L1`**), then writes the first new 1.
- States 2 and 3 write two more 1s. Then **`1L0`** steps back onto the leftmost 1 and halts.
- The blank cell never gets used.

With 2 on the tape:

Halted after 5 steps. The tape holds 5, a well-formed number.

CHALLENGE: A LONGER MACHINE

state	reads 0	reads 1
1	0R0	0R2
2	0R3	1R2
3	1R4	1R3
4	1L5	
5	0L6	1L5
6	0R1	1L6

- Type in this machine and run it with 1, 2 and 3 on the tape.
- What does it do?
- Then step through it slowly. What is each state's job?

It takes a while: with 3 on the tape, it runs for 40 steps.

CHALLENGE: THE MOST 1s

- Start with a blank tape.
- Use only states 1 and 2 (plus state 0 to halt). Fill all four cells.
- Your machine must halt. How many 1s can it leave on the tape?
- Our very first machine left four. Can you beat it?

state	**reads 0**	**reads 1**
1	?	?
2	?	?

This is called the busy beaver problem. Tibor Radó posed it in 1962.

LOOKING AHEAD: YOU'LL BUILD ONE

Later in this unit you'll write a Turing machine in C++: a program that reads a machine written down as text and runs it. Some questions to start thinking about:

How do you store one rule? A whole rule table?

The tape has no end in either direction. How do you store that?

The head can be on square −5. What happens if you use that as a vector index?

How do you read `1: 1R2 1L2` from a file?

What should your program do with a rule like `1X2`, or a blank cell?

How do you show the tape after each step?

QUESTIONS YOU MIGHT BE ASKING

Is this a real machine?

It's an idea on paper. People have built working ones for fun, even out of Lego, but nobody uses them to compute.

Why only 0 and 1?

Two symbols are enough. More symbols wouldn't let the machine compute anything new, just do it with fewer rules or steps.

Why does every rule have to move?

That's how this machine is defined. To stay on a square, move away and come back.

My machine runs forever. Is the simulator broken?

Probably not. Press Pause, then Step, and watch which rules repeat.

Why write numbers with 1s and not binary?

Adding 1 is just one more 1. Machines that count in binary need a lot more rules.

Is C++ more powerful than a Turing machine?

No. Anything a C++ program can compute, a Turing machine can too, given enough time and tape. C++ is just much easier to write.